Handoff 19: Lesson feedback engine, part 2a (ENG-04: check rules, tagged hints, hint rewording)

Date: 2026-10-27 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#25), not merged, not deployed. **No database migration. No new dependency.** Unit 1 content is at version 3; saved progress, XP and badges are unchanged.

> **How to use this file:** upload it with `build.md`, `docs/plan/lesson-feedback.md` (sections 6 to 8), handoff 15 and `backend/content/README.md`. The pull request diff is the source of truth (`build.md` section 14). Appendix A lists every new hint word for word.

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 19 (ENG-04 part 2a). Unit 1's
check messages now live in the lesson content files under stable rule
IDs (hello.two-lines and so on), and the app refuses to start if they
do not match the rules the checker uses. Hints tagged to a Python error
or a check rule are offered first, beside the error; the ladder stays
"a nudge, a bigger hint, the approach". After the approach comes a
similar example from another context, after passing "Other ways to do
it", and the hint button gets a gentle outline after two failed runs.
Every unit 1 hint was reworded (ASSESS-08). I'm attaching build.md,
docs/plan/lesson-feedback.md and this handoff. Read appendix A: check
that no hint, tagged hint or similar example gives away a full answer,
that each fits its level, and that the wording is calm and clear for
a beginner. Explain at a first-year computer engineering level.
```

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done
PR #22 (ENG-04 part 1) merged into `main`?	**Yes**, 2026-10-27 05:19 UTC; its head commit is in `origin/main`

Check	Result
Earlier ENG-04 part 2 work on any branch or commit	**None**
Branch	`feat/lesson-feedback-2a` from `origin/main` at `4f58d96`. Not stacked on PR #24 (FB-02, open)

2. Handoff record (build.md template)

Field	Value
Task	ENG-04 part 2a: named check rules and message IDs in content, validated at load; hints tagged to errors and rules; the similar example and "Other ways to do it"; the hint-button outline after two failed runs; every unit 1 hint reworded into three levels (ASSESS-08)
Human goal	When a learner is stuck, the first hint offered is about the problem they actually have. Messages can be reviewed without reading code. No hint ever adds up to the answer
Owner decisions applied	Backlog decisions 24 to 27 (friendly message first; outline after two failed runs; assumptions 1 to 3; reword unit 1 hints) and decision 5 (no word minimum, no grading change)
Repository and branch	`torontojohn/python-academy`, branch `feat/lesson-feedback-2a`
Base commit / head	Base `4f58d96`. Code, content and tests in `59039c4`; documents in the commits after it
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	Predict-first, measurements and support data, any database change, UI-05 (calling `check-runner.js` from the page), unit 2 content. These are part 2b (section 8)
Data and compatibility	No migration. Each unit 1 lesson and challenge goes from version 2 to 3; the validator (`bounded-story` v2), lesson, task and badge IDs, and badge versions are unchanged. A pass of v3 by someone who passed v2 awards nothing again; existing tests prove it. The API only **adds** fields, so the old page keeps working during the deploy
Deployment status	Not requested

Files changed

File	Why
`backend/app/content.py`	New challenge fields (`messages`, `error_hints`, `similar_example`, `other_ways`), validated at load: rule IDs start with the lesson ID and are unique across all content; hint tags are known Python errors or the challenge's own rules; one hint per tag; at most three ladder hints. `/api/lessons` gains `errorHints`, `similarExample` and `otherWays`
`backend/app/lesson_checks.py`	The checker raises rule IDs (`Fail('hello.words')`) and reads the message from the content. `RULES` lists every rule it uses; `rule_errors()` reports a rule with no message, or a message with no rule. `judge()` also returns the rule. The "copied from a hint" refusal also covers tagged hints and the similar example
`backend/app/main.py`	Refuses to start if `rule_errors()` finds anything. `/api/check` and `/api/attempts` return `rule`
`backend/content/py/u01/*.json` (5 files)	Messages (word for word as before), reworded hints, tagged hints, a similar example for each lesson, and other ways for four of them; versions 2 → 3
`backend/content/README.md`	How to write these fields
`backend/tests/test_check_rules.py` (new, 37 tests)	Rule and tag validation (below)
`backend/tests/test_honest_checks.py`	The hint audit now covers tagged hints and the similar example; the similar example must fail and every other way must pass; versions are 3
`backend/tests/test_academy.py`, `test_browser_checks.py`	Version 3 in the XP tests; `rule` added to the unit 1 response
`backend/tests/fixtures/legacy_lessons_v1.json`	`/api/lessons` as it now is: only the hints and the three new fields differ
`frontend/app/feedback.ts`	`nextHint()`: which hint to offer next. The hint panel's wording, so the snapshot shows any change
`frontend/app/adventure.tsx`	The hint panel uses `nextHint()`; counts failed runs for the outline; shows "Other ways to do it" after passing
`frontend/app/style.css`	The dashed outline (never animated) and the new panels

File	Why
`frontend/tests/hints.spec.ts` (new, 10 tests)	Unit tests of `nextHint()` and browser tests of the page
`frontend/tests/feedback.spec.ts`, `fixtures/translations.snapshot.json`, `fixtures/hello-hi-woah.json`	The hint panel wording in the snapshot; the similar example now follows the approach; `rule` in the `hi/woah` answer
`docs/plan/lesson-feedback.md`	Status line
`docs/handoffs/2026-10-27-19-lesson-feedback-2a.md` and `.pdf`	This handoff

`docs/plan/backlog.md` is **not** edited (section 10). `docs/plan/decision-log.md` is **not** edited either: open PR #24 adds chapter 15 at the same place, so a chapter here would conflict. Sections 4 to 6 of this handoff are written to be copied in as chapter 16 after #24 merges.

3. How it works

Check rules (section 6). For example, `hello.json` now holds:

```
"messages": [
  {"id": "hello.two-lines", "message": "Write two print statements with nonempty messages:
one location, then one surprise."},
  {"id": "hello.different-lines", "message": "Both lines say the same thing. Change the
second one into a surprise."},
  {"id": "hello.passed", "message": "Python printed your two different messages in order.
…"}
]
```

- The checker raises `Fail('hello.different-lines')`, and the message comes from the content.
- `lesson_checks.RULES` lists the rules the checker uses (22 across unit 1).
- If the content misses one, has one the checker never uses, or repeats an ID, the API stops at startup with a plain error, as it already does for other content mistakes.
- Rules shared by two lessons have one ID per lesson (`choices.if-else`, `adventure.if-else`), so each can be changed on its own.

The hint panel (section 7). `nextHint()` decides what the button offers:

1. If an error or failed rule is being shown and a hint is tagged for it, that hint first. Examples: a `NameError` in "Create your hero" offers "Names must be spelled exactly the same each time…"; a failed `hello.different-lines` offers its own hint. It is labelled **About this error** or **About this check**.
2. Then the ladder, in its fixed order: **A nudge**, **A bigger hint**, **The approach**.
3. Then **A similar example**: a short worked example from another context, with a note.

4. Then "That is every hint", and no answer button.

The tag comes from Python's error type (in the browser) or the `rule` the check returns. Nothing new is sent anywhere; the page only reads `rule` from the answer it already gets.

Other ways to do it. Shown only after this run passed, and gone once the code is edited.

The outline. After two failed runs in a row on the challenge card (Python stopped, or the check said "not yet"), the hint button gets a 2-pixel dashed outline. A screen reader hears "Stuck? A hint is ready whenever you want one." as its description. There is no popup, nothing moves, and nothing is animated. A pass, or moving to another card or lesson, resets the count. "Check unavailable" and "Not checked" do not count, because they are not the learner's mistake.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Tagged hints are separate from the ladder (`error_hints`), and the ladder stays untagged	Tag ladder hints and move the tagged one to the front	Moving a ladder hint forward would break "nudge, bigger hint, approach" (owner rule). Separate tagged hints satisfy both the spec ("the tagged hint first, untagged hints follow in order") and the fixed ladder	Implementer (**assumption 1**)
Messages move to content word for word	Rewording them now	Section 6's goal is reviewable messages; changing wording at the same time would mix two kinds of change	Implementer
The checker's general messages (syntax error, "run your current code again", "use the syntax taught on the cards") stay in code, with no rule ID	Give them content IDs too	They are not about one lesson's rules; they belong to the checker	Implementer (**assumption 2**)
Cross-checking content against `RULES` happens at startup, in `lesson_checks`, not in `content.py`	Put the rule list in `content.py`	`content.py` cannot import the checker (the checker imports it), and the rules belong with the code that uses them	Implementer
Versions: lesson and challenge 2 → 3; validator stays v2	Bumping the validator	The checking itself did not change; only wording did (content README: "increase `version`" for wording)	Content model
New `/api/lessons` fields are additions; `hints` stays a list of text	Turn `hints` into objects	The API and the page deploy separately; an old page must keep working with the new API	Implementer

Decision	Options considered	Reason	Who decided
The outline counts "Python stopped" and "not yet" only	Count every run that did not pass	"Check unavailable" is our fault, and "Not checked" is valid work	Implementer (**assumption 3**)
`hello` has a similar example but no "other ways"	Invent one	Any two different lines pass `hello`, so another way would just be another story; a one-line similar example cannot pass	Implementer
`rule` is returned for a pass too (`hello.passed`)	Only on failure	Simpler; the page only uses it for "not yet"	Implementer
No decision-log chapter in this PR	Add chapter 16 now	Would conflict with open PR #24	Implementer

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix, and what guards it
A similar example that handed over the answer. My first `choices` example had two plain `print` lines, which, put into the starter's two `pass` slots, make a passing program	The extended hint audit (`test_no_combination_of_hints_passes_the_check[choices]`) failed	One branch now prints using the example's own variable (`answer`), which the challenge's program does not have. The audit passes and keeps guarding it
"The approach. The approach, with other names…" The label and the hint text repeated each other in `variables` and `input`	Reading the screenshots	Reworded to "With other names: …"

Bug or near-miss	How it was found	Fix, and what guards it
A test that broke other tests. My first "refuses to start" test reloaded `app.main` inside the test run, which replaced the app object other test files had imported; 11 session tests then failed	The full suite	The test now starts the app in a separate Python process
Near-miss: a duplicate rule ID was reported on the second lesson that used it, not the first	The new test failed	The test was wrong; the behaviour is right and is now described in it

6. How it was tested

Commands and results (on `59039c4` plus the wording fix, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db .venv/bin/python -m pytest -q`	**324 passed, 10 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**334 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**93 passed** (83 before, plus 10 new)
axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA plus best practices): tagged hint shown, outlined button, similar example, other ways, each on desktop and phone	**0 violations**; no sideways scrolling at 375 px
Trial merge with PR #24 (FB-02), backend suite	No conflicts; **358 passed, 11 skipped**

What the tests cover

Required test	Where
Rule ID missing, duplicated or unused	`test_a_missing_rule_message_stops_startup`, `test_an_unused_rule_message_stops_startup`, `test_a_duplicate_rule_id_is_rejected`, `test_a_rule_id_must_belong_to_its_lesson`, `test_the_app_refuses_to_start_with_mismatched_rules` (a real app start in its own process)
Every rule really fires, with its content message	`test_every_rule_can_be_reported_with_its_message_from_the_content`: 22 sample programs, one per rule. `test_every_rule_has_a_sample_and_the_checker_names_no_other_rule` also reads the checker's source for rule names
Tags are valid	Unknown error type, wrong case, another lesson's rule, `hello.passed`, a rule that does not exist, two hints for one tag, a fourth ladder hint: each rejected at load
Each tagged hint shows first for its error	`hints.spec.ts`: for every tag in unit 1, `nextHint()` offers its hint first. On the page, a real `NameError` offers the spelling hint first, and a failed `hello.different-lines` offers its hint first; the ladder then starts at the nudge
Similar example and other ways only at the right time	The similar example only after the approach. Other ways not on load, not after a failed run, shown after passing, gone after an edit
The outline after exactly two failed runs	None at 0 or 1, present at 2 (one "not yet" plus one Python error), gone after a pass. Dashed, `animation: none`, `transition: 0s`, also with reduced motion. "Try it" runs never count. No dialog
Every reworded hint checked against its card's check	`test_no_combination_of_hints_passes_the_check` now runs the ladder, the tagged hints and the similar example's lines, alone and in every combination, with and without the starter: 28 to 121 programs per lesson, none passes. `test_the_similar_example_does_not_pass_but_every_other_way_does`
Snapshot of message templates	`translations.snapshot.json` now also holds the hint panel's wording; the diff shows only that addition
Saved progress, XP and badges untouched	The XP tests pass with version 3; the "new version does not award again" test is unchanged apart from the version number

Mocks: the browser tests mock API answers. Python, the worker and the page are real. The `hi`/`woah` answer is tied to the real API by a backend test.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
F1. `nextHint()` ignores tags	5: the three tag unit tests, and both page tests for tagged hints
F2. The outline after one failed run	1: "after exactly two failed runs, and not before"
F3. "Other ways" shown before passing	1: "the similar example comes only after the approach, and other ways only after passing"
B1. The `hello.words` message deleted from the content	**The app refuses to start**: "hello.challenge has no message for rule hello.words". Six test files cannot import the app, and 4 tests in `test_check_rules.py` fail
B2. `choices` hint 3 rewritten to give the whole answer	3: the hint audit for `choices`, plus the two fixture tests that show the content changed

7. Known limitations

- **Tagged hints only exist where unit 1 content has them.** For example, there is no `TypeError` hint, because unit 1 does not teach numbers.
- **The browser-check (ENG-03) results** give a tag only for a Python error type; their wrong-output cases have no rule IDs yet.
- **The outline count lives in the page**, so it resets on reload. That is deliberate: a gentle nudge, not a record.
- **No measurements** of whether tagged hints help (part 2b).
- **Not tested with a real screen reader,** or on Safari or Firefox.

8. Proposed part 2b (not in this PR)

Item	What
Predict-first (section 9)	Content flags per card, an optional "Predict first" step before Run, and a comparison of the prediction with the output
Measurements and support data (section 10)	"Same error on the next run", "hint level shown", "predicted correctly", with no code or message text. **Needs a database change and a privacy review**
UI-05	The lesson page reads lessons per unit and calls `check-runner.js` for lessons with browser checks; ENG-03 wrong-output results get rule IDs

Item	What
Unit 2 content (EDU-04)	Written with rules, tagged hints, a similar example and other ways from the start

9. Live checks for you after merging and deploying

Use `/preview` as a parent, or a student account.

- [ ] **The spelling hint first.** Open "Create your hero", card 5. Type `hero = "Kai"`, `tool = "map"`, `print(heor)` and run. The hint button says "Show a hint about this error". Click it: "About this error. Names must be spelled exactly the same each time…". The button then says "Show a nudge".
- [ ] **A hint about the check.** Open "Open the story", card 5. Run `print("hi")` twice. You see "Not yet. Both lines say the same thing…". The button says "Show a hint about this check", and it shows the hint about changing the second line.
- [ ] **The ladder, then the similar example.** On a fresh card 5, click the button four times: "A nudge", "A bigger hint", "The approach", then "A similar example" with a short dark code block. Then "That is every hint", and no answer button.
- [ ] **The outline.** On card 5, run something that does not pass twice. The hint button gets a dashed blue outline after the second run, not the first. It does not move or flash. A passing run removes it.
- [ ] **Other ways.** On "Create your hero", pass the challenge (for example `hero = "Kai"`, `tool = "rope"`, `print(hero, tool)`). "Other ways to do it" appears with two examples. Type in the editor and it disappears.
- [ ] **XP is unchanged.** As a student who had already passed unit 1 lessons, the XP total and badges are the same as before the deploy, and passing a lesson again adds nothing.
- [ ] **The API started.** Render → python-academy-api → Logs shows the usual startup lines, with no "Invalid course content".
- [ ] **No CSP reports** in DevTools → Console during these checks.

10. Backlog rows to change (not edited, as asked)

ID	Change
ENG-04b	proposed → in_review for **part 2a** (#25, handoff 19); add **ENG-04c (part 2b)**, proposed: predict-first, measurements and support data. UI-05 stays its own item
ASSESS-08	proposed → in_review (#25, handoff 19)
Stage table	Add "19. ENG-04 part 2a | Check rules in content, tagged hints, hint rewording | in_review (#25, handoff 19)"
Decision log	Chapter 16 from sections 4 to 6 of this handoff, after PR #24 merges

11. Questions for the owner

1. **Assumption 1:** tagged hints as separate hints, with the ladder unchanged. Is that how you pictured it?
2. **Assumption 2:** the checker's general messages stay in code without rule IDs. OK?
3. **Assumption 3:** the outline counts "Python stopped" and "not yet", but not "Check unavailable" or "Not checked". OK?
4. **Appendix A wording:** any hint you want changed? Changing one is now a content edit plus a version bump; the tests check it still gives nothing away.

12. Next concrete action

Review appendix A and the PR, merge, and run the live checks. Then decide part 2b's order relative to UI-05 and unit 2.

Appendix A. Every unit 1 hint, after this PR

The check messages themselves are unchanged, word for word, so they are not repeated here.

`hello`

Step	Text
A nudge	You need two print() lines: the first describes a place, the second something unexpected.
A bigger hint	Check each line: lowercase print, then parentheses, with your message inside matching quotes.
The approach	A pattern to complete: print("You wake up in ___.") Then add a second print() with your surprise.
About `SyntaxError`	Every opening quote needs a closing quote, and every (needs a). Check the end of each line.
About `NameError`	print is all lowercase, and words need quotes around them. Without quotes, Python reads words as names.
About `hello.words`	Each message needs words, like the name of a place or something that happens.
About `hello.different-lines`	Keep your first line as it is, and make the second one about something else that happens.
Similar example	`print("The shop opens at nine.")`. One print() shows one line of text. Your challenge uses two, with your own words.

`variables`

Step	Text
A nudge	This card needs two variables, hero and tool, each holding some text. Then print() shows them.
A bigger hint	Look at the first two lines: put your own words between the empty quotes. Then check your print(): a variable name goes inside it with no quotes.
The approach	With other names: pet = "___" stores text, and print(pet) shows it. Do the same with hero and tool.
About `NameError`	Names must be spelled exactly the same each time: hero in print() must match hero where you gave it a value, letter for letter.
About `SyntaxError`	Text values need matching quotes, like "Kai". Variable names inside print() do not.
About `variables.uses-variables`	If your output shows the words hero and tool instead of your values, remove the quotes around the names inside print().
Similar example	`pet = "Biscuit"` / `bed = "rug"` / `print(pet + " sleeps on the " + bed)`. Two variables hold text, and print() shows their values by name. Your challenge does the same with hero and tool.
Other way, after passing	An f-string puts the values straight into the sentence.
Other way, after passing	print() can take several values separated by commas, and puts a space between them.

`input`

Step	Text
A nudge	input() gives back whatever the player typed. The starter already keeps it in the variable player.
A bigger hint	Look at the comment line: replace it with a print() that joins a greeting and player.
The approach	With other names: print("Good morning, " + friend) greets whatever is stored in friend. An f-string works too: f"See you soon, {friend}".
About `NameError`	The reply is stored in player. Check that the name inside your print() is spelled exactly the same.
About `input.uses-reply`	A name typed straight into the greeting stays the same for every player. Use the variable player in the greeting instead.
Similar example	`friend = "Sam"` / `print("See you soon, " + friend)`. A variable inside a greeting. In your challenge, the value comes from input() instead of being typed into the code.
Other way, after passing	An f-string puts the reply straight into the greeting.
Other way, after passing	print() with a comma adds the space for you.

`choices`

Step	Text
A nudge	You need two branches: one when the reply is left, and one for anything else.
A bigger hint	Look at the two pass lines in the starter. Each one is the place for a print() with a discovery.
The approach	A pattern from another story: if answer == "yes": then an indented print(...), then else: and another indented print(...).
About `IndentationError`	Lines inside if and else start with four spaces. Press Tab at the start of the line to add them.
About `SyntaxError`	The if and else lines end with a colon, and comparing uses two equals signs: ==.
About `choices.different-paths`	Your two branches print the same thing. Change the words in one of them.
Similar example	`answer = input("Open the door? ")` / `if answer == "yes":` / `print("You said " + answer + ", so the door opens.")` / `else:` / `print("The door stays shut.")`. One reply, two branches, a different message in each. Your challenge compares the reply with "left".
Other way, after passing	!= means "is not equal to", so the branches can be written the other way round.

`adventure`

Step	Text
A nudge	Four parts, in order: an opening scene, a welcome that uses the name, the path question, and two different endings.
A bigger hint	Each comment in the starter marks where a print() or the if/else goes. Fill them in from top to bottom.
The approach	Patterns from earlier lessons: print("Welcome, " + name) for the welcome, and if answer == "yes": then else: for the two endings.
About `NameError`	The player's name is stored in hero, and the path in path. Use those exact names.
About `IndentationError`	Lines inside if and else start with four spaces.
About `adventure.opening-scene`	Your very first line must be a print() with a scene, before the line that asks for a name.
About `adventure.two-inputs`	Keep both input() lines from the starter: the name first, then the path.

Step	Text
Similar example	`print("The train doors open.")` / `rider = input("Ticket name? ")` / `print("All aboard, " + rider)`. A scene, a question and a welcome. Your adventure adds the path question and two different endings.
Other way, after passing	An f-string for the welcome.